Examination Room Usage Optimization Via Greedy and Exact Methods: A Vietnamese University Case Study

Minh Hieu LE [a], Surya HANJAYA [a], Trang HOANG [a] and Xuan-Bach LE [a,1]
[a] *Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology, VNU-HCM, Ho Chi Minh City, Vietnam*

Abstract. Running the exams of universities incurs significant expenses mainly because many rooms must be opened to conduct the exam at each session. Indeed, due to conservative policies and some other constraints associated with specific courses, many rooms often are not used efficiently. In contrast to previous studies on examination timetabling, which focus on joint optimization of time slots and room allocation, the current paper is dedicated to the post-examination timetabling problem of efficient room utilization. Specifically, we examine how one can minimize the number of rooms necessary for conducting the fixed examination sessions with consideration of some constraints like room capacities, campus limitations, and separation rules for courses. There are two different cases to be examined: (i) optimizing existing allocations through post-processing and (ii) allocating rooms for the exam sessions anew. We suggest a quick greedy algorithm based on a best-fit decreasing approach combined with the use of an ILP formulation to obtain optimal solutions for small-to-moderate instances. Numerical experiments performed on a real-world dataset from Ho Chi Minh City University of Technology (HCMUT), a multi-campus public university in Vietnam, show promising results, including substantial room savings and a clear scalability–optimality trade-off.

Keywords. examination timetabling, room allocation, integer linear programming, greedy heuristic, room utilization optimization

1. Introduction

Examinations are a core component to assess student performance [1,2]. However, large examination periods also create significant operational costs, especially for room allocation and invigilation [3]. In multi-campus universities, these costs grow with the number of rooms opened in each session [4]. A case study at Ho Chi Minh City University of Technology (HCMUT) [5] shows that the baseline allocation uses rooms inefficiently, with only 39.5% average utilization, while supervision requires more than 17,800 staff-hours per semester.

In practice, exam room assignments are often conservative to meet capacity, spacing, and supervision regulations [6,7]. This often leads to fragmented room usage within

[1]Corresponding Author: Xuan-Bach Le. E-mail: lexuanbach@hcmut.edu.vn. Data and artifact: `https://github.com/hieukendu/ILP`.

the same time slot. For example, two groups of 20 students from different courses may be placed in separate rooms of capacity 40 because spacing rules require empty seats between students. This fragmentation increases supervision and facility costs [8]. Improving room utilization within fixed exam schedules can therefore reduce these inefficiencies without changing academic policies or timetables [9]. In the example above, we can place both groups in the same room to use the empty seats efficiently. Exam integrity is maintained because students from different courses receive different exam papers.

The problem is further complicated in multi-campus settings. In the case of HCMUT, examinations are held concurrently at two campuses located approximately 28 km apart. Students cannot be reassigned across campuses during an examination session, while invigilators frequently travel between campuses using university-organized transportation. These constraints impose strict campus-level separation and limit the applicability of naive consolidation strategies.

Most existing research on examination timetabling focuses on the integrated assignment of exams to time slots and rooms [10,11,12]. However, less attention has been paid to exploring the problem of optimizing room use after the schedule for examinations has been devised. Practically speaking, timetables are created well ahead of time and are inflexible, while room allocations can still be adjusted. Such a situation inspires the exploration of the problem of room allocation as a self-standing task. While closely linked to the classical problem of bin packing and set partitioning models [13,14,15], this problem involves some other constraints, such as course separation, reduced examination capacities and campus-level constraints. Therefore, unlike conventional exam scheduling, in this case, time slots are already fixed, and only room use is optimized after the timetable is fixed.

This study attempts to solve the problem of reducing the number of rooms used during examinations in fixed time periods on different campuses. The examinations are done at set times and courses can be allocated in different rooms because of physical constraints. In order to ensure academic integrity, students who are taking the same course are not allowed to sit in the same room; however, those students in other courses are allowed in the same room if there is space available. All reassignments remain within the same campus.

To solve this problem, we explore two practical situations where a solution to the problem may be used. In one of these situations, we try to increase the room utilization by combining similar activities after the initial scheduling has been carried out. The second situation assumes assigning the rooms at the very beginning of the procedure while imposing the same constraints on all the activities. We show that the decision version of the problem is NP-complete and that the greedy algorithm gives a 2-approximation when room capacities are identical. We are not presenting a new approach to optimizing examinations in general; rather, we present a practical formulation and analysis of the post-timetable examination-room optimization. The results also show that the greedy method can be close to ILP at much lower runtime.

The rest of the paper is structured as follows. Section 2 reviews relevant past studies. Section 3 defines the room allocation problem with consideration of heterogeneous capacities and course-separation rules. Section 4 proposes two algorithms: a fast greedy algorithm and an ILP approach to find optimal solutions. Section 5 reports experimental results using HCMUT exam scheduling data and explores the trade-off between solution quality and running time. Section 6 concludes the study and discusses future work.

2. Related Work

Examination timetabling includes the allocation of the exam to certain time periods and venues under certain constraints. It is an operations research problem that has been widely studied due to its real-world complexity. The early literature focuses on issues such as capacity constraints of classrooms, split exams, and institutional constraints [11,12]. Recent literature focuses on actual implementation using real data and case studies [16,17].

In this more general scope, there are several papers that consider only the room allocation sub-problem where the examination time slots are pre-defined and the goal is to assign the rooms optimally. It is considered as a capacity constrained optimization problem that is very similar to the bin packing problem [14,15] and set partitioning [13]. Among other contributions, there are some researches such as those by Dammak et al. [18] and Ayob and Malik [19] that tackle similar room allocation problems. However, while the former works concentrate on the capacity-feasible room allocation, our research will be different since it will cover the optimization of room utilization after the examination timetable becomes defined and considers several practical limitations such as the campuses separation and reduced examination capacities. Most previous studies also evaluate the algorithms mainly from the computational perspective, however, our research will concentrate on the practical use of the algorithms.

Integer Linear Programming (ILP) and mixed-integer programming (MIP) have been widely used in exam timetabling because of their capability of dealing with constraints and providing an optimal solution. This is evident from previous research [20] and even current studies [21], especially on medium-scale problems. ILP has even been used for room scheduling. For example, [9] use MIP together with local search to improve room scheduling. Exact algorithms like branch and bound and decision diagrams [22,23] further highlight the strengths and limitations of ILP.

As exact algorithms do not scale well, exam timetabling problems are normally solved using heuristics and metaheuristics. Papers such as Siew et al. [24] provide overviews of adaptive and hyper-heuristics [25,26] as well as hybrid algorithms which are efficient on hard test instances. Despite that, they usually involve complicated algorithms and fine-tuning. Practically, organizations tend to use simple methods. Our room-merging greedy approach is an example of such a method which works together with ILP model.

3. Problem Formalism

We study the problem of optimizing university exam room usage. Exams are organized into independent sessions (fixed time slots) where multiple courses run in parallel, allowing each session to be optimized separately.

Let $\mathscr{T}$ denote the set of examination sessions. For each session $t \in \mathscr{T}$, let $\mathscr{R}_t$ be the set of rooms available during that session. Each room $j \in \mathscr{R}_t$ has a physical capacity c_j and an exam capacity $\hat{c}_j \leq c_j$, which accounts for spacing and supervision constraints. At HCMUT, exam capacity is typically about half of the physical capacity ($\hat{c}_j \approx 0.5 c_j$).

Let $\mathscr{K}$ denote the set of courses. Each course is divided into one or more *candidate groups*, indexed by $i \in \mathscr{I}$. Each group represents a cohort of students that must be seated together, has size s_i, belongs to exactly one course $k(i) \in \mathscr{K}$, and participates in exactly one examination session $t(i) \in \mathscr{T}$.

Candidate groups are *indivisible*: all students in a group must be assigned to a single examination room, and any reassignment moves the group as a whole. Groups from different courses may share the same room if space allows.

An assignment to room j is *feasible* if: (i) total assigned students do not exceed the room capacity, (ii) per-course students do not exceed the exam capacity, and (iii) no two groups belong to the same course. Formally,

$$\sum_{i:\, i\to j} s_i \le c_j, \qquad \sum_{i:\, i\to j,\, k(i)=k} s_i \le \hat{c}_j, \quad \forall k \in \mathscr{K}, \qquad \sum_{i:\, i\to j,\, k(i)=k} 1 \le 1, \quad \forall k \in \mathscr{K}. \tag{1}$$

A room is *open* in session t if at least one candidate group is assigned to it, and *closed* otherwise. Since each room incurs a fixed cost, the objective for each session is to minimize the number of open rooms by packing candidate groups from different courses into shared rooms while respecting all feasibility constraints.

We consider two closely related optimization scenarios:

1. **Room merging (post-processing).** Each session begins with a feasible room assignment, typically generated by the university's existing system. The goal is to reduce the number of rooms in use by reassigning entire student groups among the already assigned rooms. No new rooms can be added, and any room left empty is closed. All reassignments must respect capacity limits and course-separation rules.
2. **Room assignment from scratch.** No initial assignment is provided. Candidate groups are assigned directly to rooms so as to minimize the number of open rooms, subject to physical capacity, exam capacity, indivisibility, and course-separation constraints.

The two scenarios above reflect common practice. The post-processing scenario allows for small improvements to be made to existing systems with less disruption, while the from-scratch scenario provides an ideal scenario in which it becomes easier to understand the theoretical maximum efficiency that can be achieved under the same conditions.

4. Proposed Methods

In this section, we discuss two techniques: one is a scalable greedy heuristic that can be used for real-life applications (Section 4.1), while the other is an integer linear programming (ILP) formulation which provides optimal benchmarks (Section 4.2).

4.1. Greedy Assignment Algorithm

The greedy algorithm (Algorithm 1) processes each examination period such that the minimum number of rooms is used. This is done by packing groups of candidates from different courses in common rooms without violating any room capacity and course separation constraints. Two versions of the algorithm are considered here based on problem statements presented in Section 3.

Case 1: Room merging (post-processing). An initial assignment A_0 is provided, where each candidate group is assigned to a room. The objective is to keep only a subset

Algorithm 1: Greedy assignment for exam session t

Input: Candidate groups $\mathscr{G}_t$ with sizes s_i, courses $k(i)$, original rooms $\text{room}(i)$, exam capacities $\hat{c}_i$; available rooms $\mathscr{R}_t$ with physical capacities c_j and exam capacities $\hat{c}_j$; optional initial assignment A_0

Output: Assignment A and open rooms $\mathscr{O}$

$\mathscr{G}_t \leftarrow \text{SortDesc}(\mathscr{G}_t, s)$

if $A_0 \neq \emptyset$ **then**

  `// Case 1: room merging (post-processing)`

  $A \leftarrow \emptyset$; $\mathscr{O} \leftarrow \emptyset$; bins $\leftarrow [\,]$

  foreach $i \in \mathscr{G}_t$ **do**

    find a feasible bin b with sufficient remaining physical capacity, with group size $s_i \leq \hat{c}_b$, and no course conflict

    if *no such bin exists* **then**

      open a new bin using $\text{room}(i)$ as target

      add it to $\mathscr{O}$

    assign i to the selected bin and update its load

else

  `// Case 2: direct assignment from scratch`

  $\mathscr{R}_t \leftarrow \text{SortDesc}(\mathscr{R}_t, c)$ `// Open large rooms first.`

  $A \leftarrow \emptyset$; $\mathscr{O} \leftarrow \emptyset$

  foreach $i \in \mathscr{G}_t$ **do**

    find a feasible room $j^\star \in \mathscr{O}$ that minimizes the remaining capacity

    if *no such room exists* **then**

      open the next room $j \in \mathscr{R}_t$ and set $\mathscr{O} \leftarrow \mathscr{O} \cup \{j\}$

      $j^\star \leftarrow j$

    $A \leftarrow A \cup \{(i \rightarrow j^\star)\}$

return $A, \mathscr{O}$

of these rooms open by merging groups into them. No new rooms may be opened, and rooms left empty are closed.

The algorithm treats each group as indivisible and applies a best-fit decreasing strategy. Groups are sorted in descending order of size and assigned to the first feasible open room. A room is feasible for such a group if: (i) it has enough remaining physical capacity, (ii) its exam capacity is not violated, (iii) it does not contain a group from the same course. If no such room exists, the group's original room is retained. Rooms not selected as targets are implicitly closed.

Case 2: Direct assignment from scratch. When no initial assignment exists, rooms are selected directly from $\mathscr{R}_t$. Rooms are first sorted in descending order of physical capacity and opened greedily as needed. Candidate groups are processed in descending order of size. Each group is assigned to a feasible open room that minimizes the remaining capacity. If no open room is feasible, the next unused room in $\mathscr{R}_t$ is opened.

Discussion. The post-processing variant limits room usage by selecting a subset of initially assigned rooms and closing the rest, never opening new ones. In contrast, the

direct-assignment variant opens rooms from the available pool in descending order of capacity and assigns groups greedily. Both approaches are fast and scalable but do not guarantee optimal solutions, as early decisions may block better overall assignments. This trade-off between optimality and efficiency is well known in bin packing and exam timetabling literature [14,15,11,25].

4.2. Integer Linear Programming Formulation

We formulate the exam room assignment problem as an ILP, which yields optimal solutions and serves as a benchmark for comparison. As in the heuristic approach, the formulation distinguishes between post-processing (room merging) and full assignment.

Decision variables. For each candidate group i and room j available in the same exam session, we introduce a binary assignment variable:

$$x_{ij} = \begin{cases} 1, & \text{if candidate group } i \text{ is assigned to room } j, \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

For each room j, we define a binary activation variable:

$$y_j = \begin{cases} 1, & \text{if room } j \text{ is kept open,} \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

Optimization scenarios. The ILP can be instantiated in two modes.

- **Room merging (post-processing).** An initial assignment A_0 is given, together with the set of rooms $\mathscr{R}_0$ that are used in A_0. In this mode, no new rooms may be opened and the solver may close rooms in $\mathscr{R}_0$ and reassign their groups. Formally, we restrict feasible rooms to $\mathscr{R}_0$ by enforcing:

$$y_j = 0 \; \forall j \notin \mathscr{R}_0 \quad \text{and} \quad x_{ij} = 0 \; \forall i, \; \forall j \notin \mathscr{R}_0. \tag{4}$$

  The activation variables y_j for $j \in \mathscr{R}_0$ remain decision variables, allowing the ILP to select a minimum-cardinality subset of rooms to keep open.
- **Room assignment from scratch.** No initial assignment is given. All variables $x_{ij} \in \{0,1\}$ and $y_j \in \{0,1\}$ are optimized freely, allowing rooms to be opened or closed as needed. This setting yields the minimum possible number of rooms for each session under the full set of constraints.

Objective. In both cases, we want to minimize the number of open rooms:

$$\min \sum_j y_j. \tag{5}$$

Assignment constraints. Each candidate group is assigned to exactly one room:

$$\sum_j x_{ij} = 1, \quad \forall i. \tag{6}$$

Exam constraints. Two constraints are imposed: (i) candidate groups belonging to the same course cannot share the same examination room, and (ii) each assigned group must satisfy the room's examination capacity.

$$\sum_{i:k(i)=k} x_{ij} \le y_j \quad \forall j \in \mathscr{R}, \forall k \in \mathscr{K} \quad \text{and} \quad s_i x_{ij} \le \hat{c}_j \quad \forall i \in \mathscr{G}, \forall j \in \mathscr{R}. \tag{7}$$

Physical constraints. The total number of students in a room must satisfy:

$$\sum_i s_i x_{ij} \le c_j y_j, \quad \forall j. \tag{8}$$

Room activation constraints. Groups can only be assigned to open rooms:

$$x_{ij} \le y_j, \quad \forall i, j. \tag{9}$$

For a session with n groups, m rooms, and $|\mathscr{K}|$ courses, the model has $O(nm)$ binary variables and $O(nm + m|\mathscr{K}|)$ constraints. Solution time grows with size, but the ILP remains tractable for moderate sessions and provides optimal benchmarks.

4.3. Complexity Analysis

We study the decision version of the exam room allocation problem:

Problem 4.1: Exam Room Allocation Decision Problem (ERAD)

The input consists of a set of candidate groups $\mathscr{G}$ and a set of available rooms $\mathscr{R}$. Each group i has size s_i and course label $k(i)$. Each room j has a physical capacity c_j and an examination capacity $\hat{c}_j$.
Given a bound B on the number of rooms, determine whether all groups can be assigned to at most B rooms while satisfying the physical capacity, examination capacity, and course-separation constraints.

Theorem 1. *The Exam Room Allocation Decision Problem is NP-complete.*

Proof. ERAD is in NP since a candidate assignment can be verified in polynomial time by checking the assignment and all constraints.

To prove NP-hardness, we reduce from the Bin Packing decision problem. Given items with sizes $a_1, \ldots, a_n$, bin capacity C, a limit of B bins, construct an ERAD instance as follows. For each item i, create a group g_i with size $s_i = a_i$. Create B rooms, each with capacities $c_j = \hat{c}_j = C$. Assign each group a distinct course label so that course-separation constraints are inactive.

A packing into at most B bins exists iff the ERAD instance admits an assignment using at most B rooms. Hence Bin Packing reduces to ERAD in polynomial time, implying ERAD is NP-complete. □

Theorem 2. *If all rooms have identical capacity C, the direct greedy assignment algorithm is a* 2*-approximation. That is, ALG* $\le 2\,OPT$.

Table 1. Overview of the HCMUT centralized examination period (Semester 251).

Statistic	Overall	Campus 1	Campus 2
Total candidate demand	115,641	39,245	76,396
Total room usages	3,392	1,377	2,015
Estimated supervision workload (hours)	17,808	7,807	10,001
Number of examination rooms	202	98	104
Average exam capacity per room	42	31	53
Average physical capacity per room	80	58	100
Average students per room	34	29	38
Avg physical utilization (%)	**39.5**	**42.2**	**37.7**

Proof. Let ALG be the number of rooms opened by the greedy algorithm, $L_1, \ldots, L_{ALG}$ their loads, and $S = \sum_i s_i$ the total number of candidates. Since each room has capacity C, any solution using OPT rooms satisfies:

$$S \leq OPT \cdot C.$$

In the greedy process, a new room is opened only if the current group of size s cannot fit into any existing room. Hence for every open room R_ℓ:

$$L_\ell + s > C.$$

In particular, for the most recently opened room R_t, $L_t + s > C$. After opening a new room R_{t+1} and assigning the group to it, we get: $L_t + L_{t+1} > C$. Thus every pair of consecutive rooms has combined load greater than C, so:

$$S = \sum_{j=1}^{ALG} L_j > \left\lfloor \frac{ALG}{2} \right\rfloor C.$$

Combining this with the inequality $S \leq OPT \cdot C$ yields:

$$ALG \leq 2\,OPT.$$

Thus the greedy algorithm is a 2-approximation. □

The NP-completeness result shows that optimal solutions are hard to compute in general, which motivates the use of heuristics. The approximation theorem guarantees that the greedy algorithm uses at most twice the minimum number of rooms under identical capacities. This guarantee is restricted to the identical-capacity direct-assignment setting and should be interpreted as a theoretical baseline rather than a direct bound for the heterogeneous room capacities considered in our case study.

5. Case Study

We evaluate the proposed methods using a real-world case study from Ho Chi Minh City University of Technology (HCMUT) [5]. In semester 251 (August–December 2025), Ho

Chi Minh City University of Technology (HCMUT) had an examination period which lasted for 19 days and involved up to five sessions per day while running simultaneously on two campuses located 28 km apart.

Traditionally, courses have been run at both campuses, where examinations are held simultaneously for each of them. The cost of conducting examinations is significant since there are two invigilators needed for each room for each session along with 5% contingency for the number of rooms allocated. Invigilators often commute from one campus to another via university buses, making it even more complex.

Table 1 shows the major features of the examination period. Overall, 81 session slots have been allocated, giving 3,392 room allocations in the baseline allocation. It corresponds to the approximate supervisory effort of 17,808 staff-hours, close to two years of work. However, despite the massive number, the physical utilization of the rooms is still rather small. Rooms have various sizes, and due to space and supervision limitations, their capacity is usually only 50% of physical capacity.

The physical usage of room r and the average physical usage during the whole examination period are given as:

$$u_r^{\text{phys}} = \frac{n_r}{c_r} \quad \text{and} \quad \bar{u}^{\text{phys}} = \frac{1}{|\mathscr{R}^{\text{open}}|} \sum_{r \in \mathscr{R}^{\text{open}}} u_r^{\text{phys}}. \tag{10}$$

where n_r is the number of students assigned to room r, c_r is its physical capacity, and $\mathscr{R}^{\text{open}}$ the set of all rooms that are open during the examination period.

Under the baseline allocation, average physical room utilization is only 39.5%, and even lower at Campus 2 (37.7%). This low utilization leaves substantial room for cost reduction by co-locating candidate groups from different courses within the same room, while respecting capacity and supervision constraints.

5.1. Experimental Setup

The input data consist of official room allocation plans for each examination session, including group sizes, course identifiers, room capacities, and campus information. Examinations are handled separately by campus: within each session, rooms are assigned only to candidate groups from the same campus, and no room is shared across campuses.

Both methods are evaluated relative to the baseline allocation produced by the university scheduling system. We consider three evaluation metrics: (i) room reduction with respect to the baseline, (ii) average physical room utilization after merging, and (iii) computational running time.

For the ILP approach, we use the CBC solver via the PuLP interface [27]. Each examination day (up to five sessions) is solved per session with a 15-minute time limit. A day is marked as a *timeout* if any session exceeds the limit, in which case the best incumbent solution is retained. Experiments run on a Mac mini (Apple M4, 10 cores, 16 GB RAM). Days are processed independently to reflect operational constraints. To improve reproducibility, the full exam-room instance dataset used in this study is provided as supplementary material. The source code, implementation scripts, and usage instructions are publicly available in the project repository: `https://github.com/hieukendu/ILP`. The released dataset contains the fields required by the optimization model, including examination session identifiers, campus identifiers, course identi-

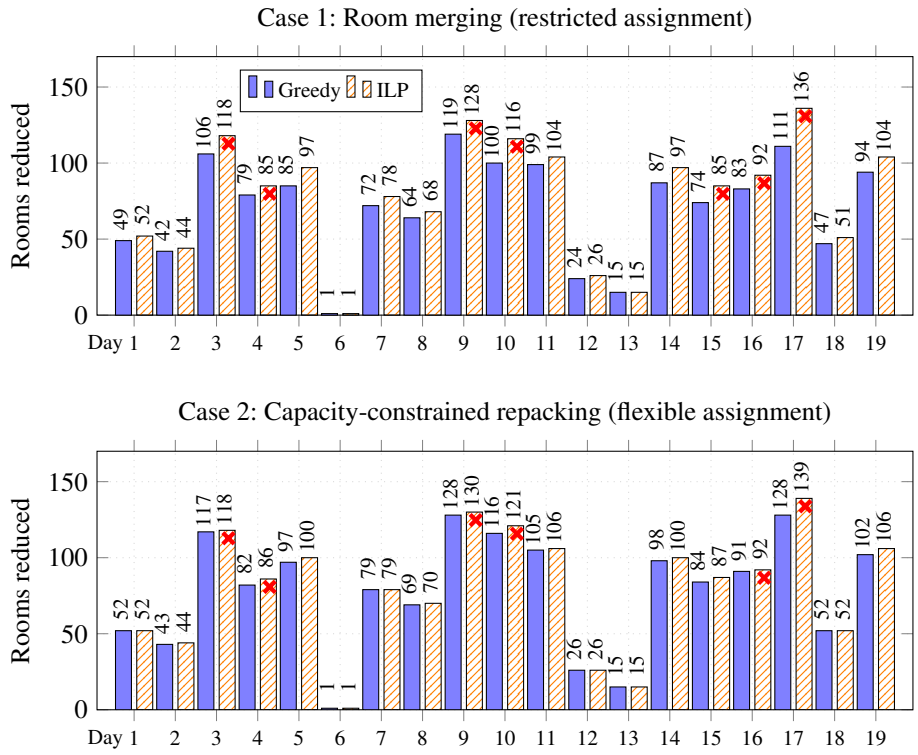

Figure 1. Room reduction under restricted (Case 1) and flexible (Case 2) regimes. Days marked with * indicate ILP timeouts (15 minutes); feasibility is still preserved.

fiers, room identifiers, group sizes, physical room capacities, and examination capacities. Student-level records and personally identifiable student information are not used by the model and are not included in the released dataset.

The ILP models were implemented in Python 3.12.7 using PuLP 3.3.0 and solved with CBC 2.10.3 through the `PULP_CBC_CMD` interface. Each exam session was solved independently with a 900-second time limit. CBC solver output was enabled during execution. The relative MIP gap, absolute MIP gap, thread count, presolve, and cutting-plane parameters were not explicitly modified and were therefore left at the default CBC/PuLP settings. When CBC returned an optimal solution, the corresponding objective value and assignment were recorded. When the time limit was reached, the best feasible incumbent solution, if available, was retained and the session was marked as a timeout. In the post-processing setting, the baseline allocation is assumed feasible; therefore, if a group cannot be merged into any feasible target room, it remains assigned to its original room. In the from-scratch setting, the greedy algorithm opens additional rooms from the same session and campus as needed. If no feasible room remains, the instance is flagged as infeasible and the original operational allocation is retained. No cross-campus reassignment is allowed in any setting.

Table 2. Room usage and room utilization comparison (best results in **bold**).

	Base	Greedy1	ILP1	Greedy2	ILP2
Room usages	3,392	2,041	1,895	1,907	**1,868**
Reduction (%)	–	39.8%	44.1%	44.8%	**44.9%**
Avg. phys. util.	39.5%	**63.9%**	60.3%	51.2%	56.1%

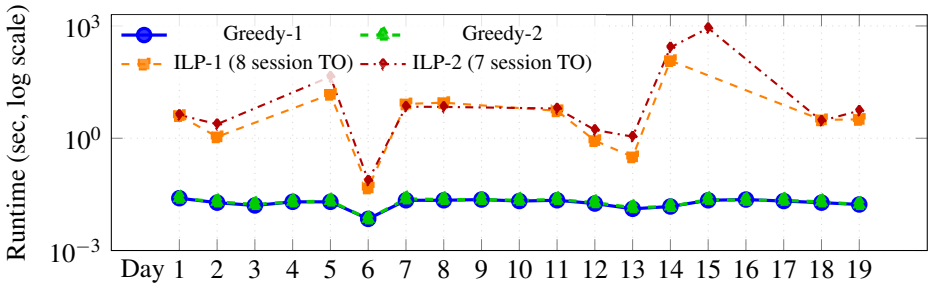

Figure 2. Runtime comparison between the Greedy and ILP approaches (Cases 1–2) on a logarithmic y-axis; days with session timeouts are omitted.

5.2. Evaluation Results

This section evaluates the effectiveness and computational performance of the proposed greedy heuristic and ILP-based optimization under two assignment regimes. Case 1 restricts assignments to room merging, while Case 2 allows flexible, capacity-constrained reassignment. We examine three aspects: room reduction relative to the baseline, average physical room utilization, and runtime performance. Aggregate results for the first two metrics are summarized in Table 2, with day-level trends shown in Figure 1.

Room reduction and utilization. Figure 1 shows the number of rooms reduced on each examination day compared to the baseline scenario. For all 19 scheduling instances, both heuristics provide significant savings in the number of required rooms, which confirms the over-provisioning problem for the base schedules.

For the first case, the greedy algorithm removes 1,351 rooms (39.8%), while the ILP model manages to remove more rooms by 1,497 rooms (44.1%). These results can be seen from Table 2, which shows the advantage of the ILP model by 4.3 percentage points. Among all methods, the Case 1 greedy heuristic attains the highest average physical room utilization (63.9%), representing a 1.6 times increase over the baseline utilization of 39.5%, as its aggressive local packing strategy prioritizes filling rooms as tightly as possible.

In Case 2, both methods benefit from the added flexibility of capacity-constrained repacking. The greedy heuristic reduces 1,485 rooms (44.8%), while ILP achieves 1,524 rooms (44.9%). The 0.1% gap (Table 2) shows that with flexible reassignment the greedy approach is nearly as effective as ILP.

Runtime performance and scalability. Figure 2 compares runtimes on a logarithmic scale. The greedy heuristic is consistently fast and stable across all sessions and both

regimes, with a total runtime of about 0.37 seconds over 19 days in Case 1 and a comparable time in Case 2, demonstrating strong scalability.

In contrast, ILP runtimes are highly variable and sensitive to both instance structure and assignment regime. While many sessions solve within seconds, others exhibit sharp runtime spikes, with several exceeding hundreds of seconds and reaching the 15-minute limit. Overall, ILP incurs 8 session timeouts in Case 1 and 7 in Case 2. Although the best feasible solutions before timeout remain valid, these results highlight the limited predictability of exact optimization in large, tightly constrained settings. Case 2 increases runtime variability due to the larger reassignment search space.

Comparative analysis and implications. Overall, the results highlight a clear trade-off between solution quality and efficiency. ILP provides a modest advantage under restricted assignment, improving room reduction by 4.3% in Case 1, but this shrinks to just 0.1% in Case 2. By contrast, the greedy heuristic is orders of magnitude faster and incurs no timeouts in either regime, making it more practical for large-scale deployment.

These results show that flexible allocation skews the equilibrium towards the greedy algorithm. With repacking enabled, the greedy method achieves close-to-optimal reduction of rooms at a very low cost, making it highly appropriate for extensive applications that require efficient planning. ILP is still highly useful for evaluation purposes; yet, its small gains may not be worth the additional cost.

6. Conclusion and Future Work

We explore post-hoc examination room optimization under restricted and flexible assignment settings by comparing a greedy heuristic and ILP formulations. In the restricted assignment setting, ILP offers a 4.3% improvement, while in flexible reassignment the improvement drops to 0.1%. On the other hand, the greedy heuristic remains significantly faster and has no timeouts. Our findings suggest that flexible assignment makes exact optimization less beneficial in practice. Further research will be devoted to extensions to multi-objective case, dynamic and stochastic settings and applications of developed algorithms in real world examination scheduling systems. We will also test the suggested techniques using examination datasets from different universities.

Acknowledgements

This research is funded by Murata Science and Education Foundation under grant number 26VH05. We acknowledge the support of time and facilities from Ho Chi Minh City University of Technology (HCMUT), VNU-HCM for this study.

References

[1] Abdul-Rahman S, Burke EK, Bargiela A, McCollum B, Özcan E. A constructive approach to examination timetabling based on adaptive decomposition and ordering. Annals of Operations Research. 2014;218:3-21.

[2] Burke EK, Petrovic S. Recent research directions in automated timetabling. European Journal of Operational Research. 2002;140(2):266-80.

[3] Qu R, Burke EK, McCollum B, Merlot LTG, Lee SY. A survey of search methodologies and automated system development for examination timetabling. Comput Oper Res. 2009;37(3):397-409.

[4] Gaspero LD, Schaerf A. Tabu Search Techniques for Examination Timetabling. In: Selected Papers from the Third International Conference on Practice and Theory of Automated Timetabling III. PATAT '00. Berlin, Heidelberg: Springer-Verlag; 2000. p. 104—117.
[5] Ho Chi Minh City University of Technology. Ho Chi Minh City University of Technology (HCMUT); 2025. Accessed: 2025-01. https://www.hcmut.edu.vn.
[6] Ahuja RK, Orlin JB, Tiwari A. A greedy genetic algorithm for the quadratic assignment problem. Comput Oper Res. 2000;27:917-34.
[7] Wasfy A, Aloul FA. Solving the University Class Scheduling Problem Using Advanced ILP Techniques. In: GCC Conference; 2007. .
[8] Ceschia S, Di Gaspero L, Schaerf A. Design, Engineering, and Experimental Analysis of a Simulated Annealing Approach to the Post-Enrolment Course Timetabling Problem. Comput Oper Res. 2011 04;39.
[9] Lemos A, Melo FS, Monteiro PT, Lynce I. Room Usage Optimization in Timetabling: A Case Study at Universidade de Lisboa. Operations Research Perspectives. 2019;6:100092.
[10] Schaerf A. A survey of automated timetabling. Artificial Intelligence Review. 1999;13:87-127.
[11] Kahar MNM, Kendall G. The Examination Timetabling Problem at Universiti Malaysia Pahang: Comparison of a Constructive Heuristic with an Existing Software Solution. European Journal of Operational Research. 2010;207(2):557-65.
[12] Bergmann LK, Fischer K, Zurheide S. A Linear Mixed-Integer Model for Realistic Examination Timetabling Problems. In: PATAT; 2014. p. 82-101.
[13] Ryan DM, Foster BA. An Integer Programming Approach to Scheduling. In: Computer Scheduling of Public Transport. North-Holland; 1981. p. 269-80.
[14] Coffman EG, Garey MR, Johnson DS. Approximation Algorithms for Bin Packing: A Survey. In: Hochbaum DS, editor. Approximation Algorithms for NP-Hard Problems; 1996. p. 46-93.
[15] Delorme M, Iori M, Martello S. Bin Packing and Cutting Stock Problems: Mathematical Models and Exact Algorithms. European Journal of Operational Research. 2016;255(1):1-20.
[16] Sigurpálsson ÁÖ. Examination timetable modelling at the University of Iceland [Master's thesis]. Reykjavík, Iceland: University of Iceland; 2017.
[17] Schininà R. An Integer Programming Model for Timetabling at the University of Groningen [Bachelor's thesis]. University of Groningen; 2024.
[18] Dammak A, Elloumi A, Kamoun H. Classroom Assignment for Exam Timetabling. Advances in Engineering Software. 2006;37(10):659-66.
[19] Ayob M, Malik A. A New Model for an Examination-Room Assignment Problem. International Journal of Computer Science and Network Security. 2011;11(10):255-62.
[20] Boland NL, Hughes BD, Merlot LTG, Stuckey PJ. New integer linear programming approaches for course timetabling. Computers & Operations Research. 2008;35(7):2209-33.
[21] Laisupannawong T, Sumetthapiwat S. An ILP Model for the Examination Timetabling Problem: A Case Study of a University in Thailand. Advances in Operations Research. 2024;2024:1-17.
[22] González JE, Ciré AA, Lodi A, Rousseau LM. Integrated IP and Decision Diagram Search Tree with an Application to the Maximum Independent Set Problem. Constraints. 2020;25(1):23-46.
[23] Amaldi E. Branch and Bound Method; 2019. Accessed: 2026-01-21. Lecture notes, Foundations of Operations Research, Politecnico di Milano.
[24] Siew ESK, Sze SN, Goh SL, Kendall G, Sabar NR, Abdullah S. A Survey of Solution Methodologies for Exam Timetabling Problems. IEEE Access. 2024;12:41479-98.
[25] Burke EK, Bykov Y, Newall JP, Qu R. A Graph-based Hyper-heuristic for Educational Timetabling Problems. European Journal of Operational Research. 2007;176(1):177-92.
[26] Soghier A, Qu R. Adaptive Selection of Heuristics for Assigning Time Slots and Rooms in Exam Timetables. Applied Intelligence. 2013;39(2):438-50.
[27] Mitchell S, O'Sullivan M, Dunning I. PuLP: A Linear Programming Toolkit for Python; 2011. Available from: https://coin-or.github.io/pulp/.